

ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

канд. техн. наук , доцент
должность, уч. степень, звание

подпись, дата

А. А. Попов
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

по курсу: Программирование встроенных приложений

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4131з

подпись, дата

М.Д. Быстров
инициалы, фамилия

Санкт-Петербург 2026

Цель работы

Освоение принципов автоматного программирования как механизма организации управления периферийными подсистемами микроконтроллера. Изучение принципов работы с интерфейсами: универсальным асинхронным приёмопередатчиком UART;

Задание на лабораторную работу

Данные для варианта 4:

№	Описание задачи
4	Сравнение числа букв. На ленте расположено слово из букв а и б, например: abbab. Требуется определить, какой символ повторяется чаще, и записать его через одну ячейку после окончания слова. Исходное слово сохранять не обязательно. Для приведенного примера результат может иметь вид: xxxxx□b. Если количество букв а и б в слове одинаково, в указанную ячейку ничего не записывается. Пример графа МТ: <pre> graph LR q0((q0)) -- "a; x, L" --> q2((q2)) q0 -- "b; x, L" --> q3((q3)) q0 -- "□; □, R" --> q1((q1)) q2 -- "x; x, R" --> q2 q2 -- "b; b, R" --> q2 q2 -- "□; □, R" --> q4((q4)) q3 -- "x; x, R" --> q3 q3 -- "a; a, R" --> q3 q3 -- "a; x, R" --> q1 q3 -- "□; □, R" --> q5((q5)) q1 -- "x; x, R" --> q1 q1 -- "□; □, S" --> q6(((q6))) q4 -- "□; b, S" --> q6 q5 -- "□; a, S" --> q6 q0 -- "□; ~, L" --> q0 </pre>

Граф автомата

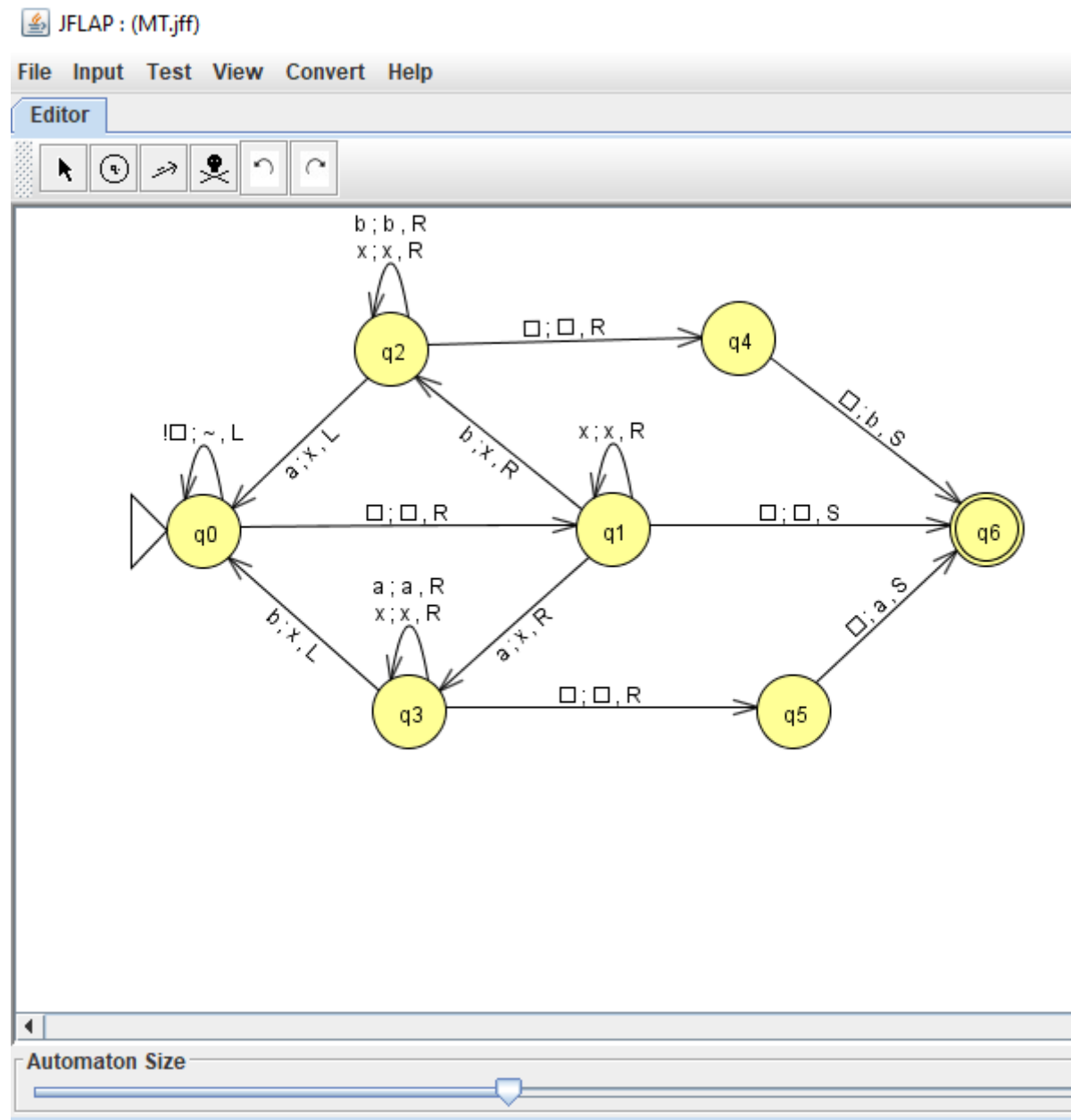


Рис.1. Граф автомата в JFLAP.

Описание состояний и переходов автомата:

- q0 – Начальное состояние, смещаемся влево до тех пор, пока не найдем пустой символ, тогда смещаемся вправо и переходим в q1. По сути, в этом состоянии идет поиск начала слова.
- q1 – Определение буквы. Если “x” – буква обработана, движемся по ленте дальше вправо. Если “a” – помечаем ее как “x”, движемся по ленте дальше вправо и переходим в состояние q3. Для “b” – тоже самое, что и для “a”, только переходим в состояние q2. Если пустой символ (конец слова) – движемся по ленте вправо и переходим в финальное состояние q6. Если мы нашли конец слова в этом состоянии, считается, что кол-во “a” и “b” одинаковое.
- q2 – “подсчет” “b” – движемся по ленте вправо, пока находим “b” или “x”. Если находим “a” – помечаем как “x”, движемся по ленте влево и возвращаемся в

состояние q0 (для возвращения к началу слова). Если нашли пустой символ, движемся по ленте вправо и переходим в состояние q4

- q3 – “подсчет” “а”. Все тоже самое, что и с q2 для “b”, только вместо “b” мы остаемся в q3 при нахождении “а”, а в q0 переходим при нахождении “b”.
- q4 – запись результата “b”. Записываем “b” вместо пустого символа. Переходим в состояние q6. Считаем, что больше всего в слове было букв “b”.
- q5 – запись результата “а”. Тоже, что и в q4, только в качестве результата – “а”.
- q6 – конечное состояние.

Результат работы МТ

```
wr<1F0:F>_aab____aab____
wr> ok
rd<1F0:F>
_aab____aab____
tm<1:7>1F2:020
TM1>step=4 addr=1F2 sym='a'
TM2>step=0 addr=1FA sym='a'
TM1>step=5 addr=1F1 sym='a'
TM2>step=1 addr=1F9 sym='a'
TM1>step=6 addr=1F0 sym='_'
TM2>step=2 addr=1F8 sym='_'
TM1>step=7 addr=1F1 sym='x'
TM2>step=3 addr=1F9 sym='x'
TM1>step=8 addr=1F2 sym='a'
TM2>step=4 addr=1FA sym='a'
TM1>step=9 addr=1F3 sym='x'
TM2>step=5 addr=1FB sym='x'
TM1>step=10 addr=1F2 sym='a'
TM2>step=6 addr=1FA sym='a'
TM1>step=11 addr=1F1 sym='x'
TM2>step=7 addr=1F9 sym='x'
TM1>step=12 addr=1F0 sym='_'
TM2>step=8 addr=1F8 sym='_'
TM1>step=13 addr=1F1 sym='x'
TM2>step=9 addr=1F9 sym='x'
TM1>step=14 addr=1F2 sym='x'
TM2>step=10 addr=1FA sym='x'
TM1>step=15 addr=1F3 sym='x'
TM2>step=11 addr=1FB sym='x'
TM1>step=16 addr=1F4 sym='_'
TM2>step=12 addr=1FC sym='_'
TM1>step=17 addr=1F5 sym='a'
TM2>step=13 addr=1FD sym='a'
TM1>stop addr=1F5 sym='a'
TM2>stop addr=1FD sym='a'
rd<1F0:F>
_xxx_a__xxx_a__
```

Рис. 2. Результат работы 2-х МТ.

```

wr<1F0:F>_aaa____abb____
wr> ok
rd<1F0:F>
_aaa____abb____
tm<1:7>1F2:020
K13
TM1>step=1 addr=1F2 sym='a'
TM2>step=1 addr=1FA sym='a'
K13
TM1>step=2 addr=1F1 sym='b'
TM2>step=2 addr=1F9 sym='a'
TM1>step=3 addr=1F0 sym='b'
TM2>step=3 addr=1F8 sym='_'
TM1>stop addr=1F0 sym='e'
TM2>step=4 addr=1F9 sym='x'
TM2>step=5 addr=1FA sym='x'
TM2>step=6 addr=1F9 sym='x'
TM2>step=7 addr=1F8 sym='_'
TM2>step=8 addr=1F9 sym='x'
TM2>step=9 addr=1FA sym='x'
TM2>step=10 addr=1FB sym='x'
TM2>step=11 addr=1FC sym='_'
TM2>step=12 addr=1FD sym='b'
TM2>stop addr=1FD sym='b'
rd<1F0:F>
eaaa____xxx_b_
rd<1F0:F>
eaaa____xxx_b_

```

Рис. 3. Обработка нажатия кнопки.

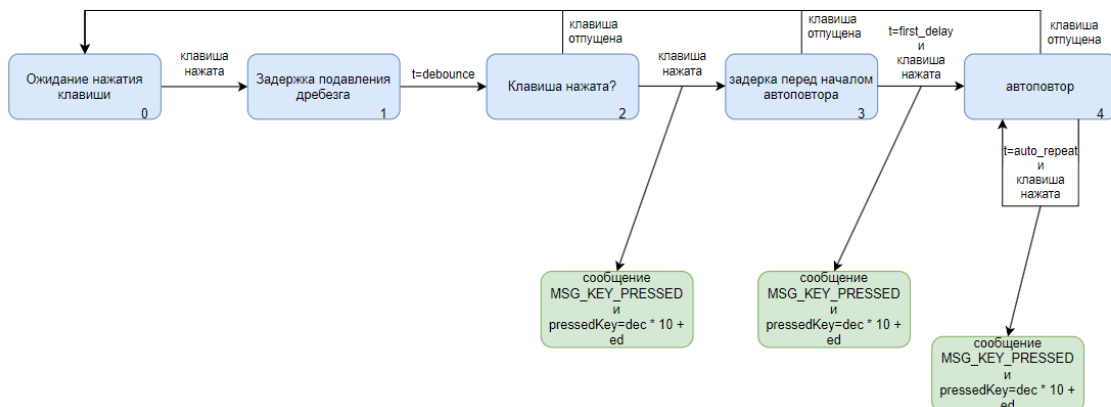


Рис 4. Граф программного автомата main.c (нажатие кнопки)

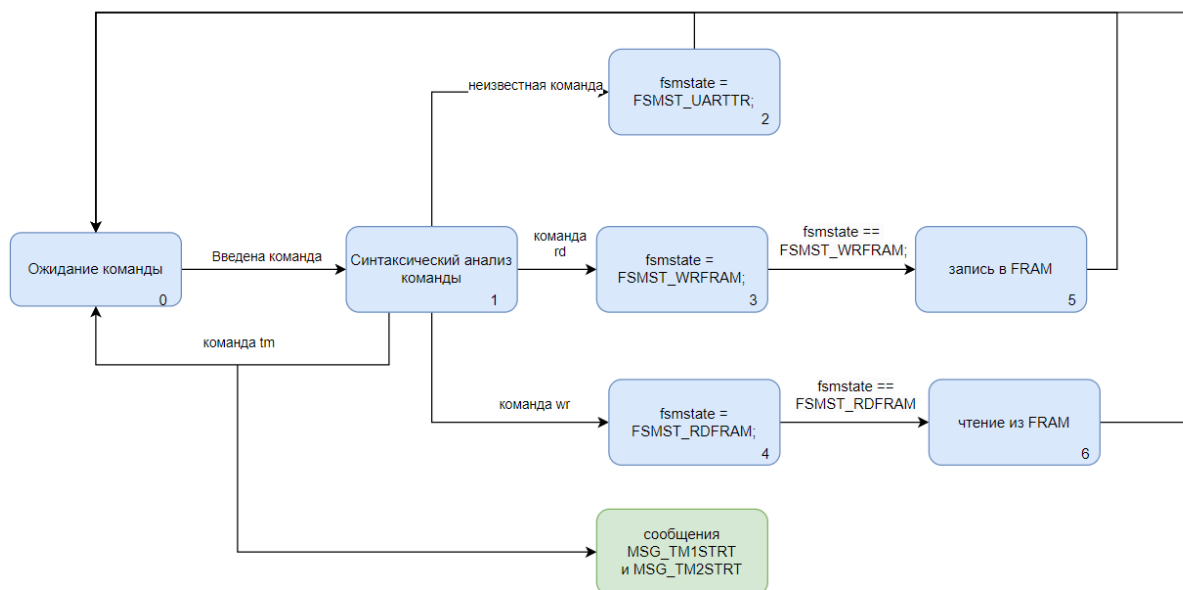


Рис. 5. Граф программного автомата fsm_main.c

Код реализации автомата

```

int ProcTm(uint8_t rsymb,uint8_t* wrsymb)
{
    if (curraddrtm < TM1_START || curraddrtm > TM1_START + 8) {
        curraddrtm = TM1_START;
        *wrsymb = 'e';
        fsmstate = FSMST_Q6;
        printf("TM1 wrong address %03X",curraddrtm);
        return -1;
    }

    static int stepcnt=0;
    *wrsymb=rsymb;
    stepcnt++;
    switch(fsmstate)
    {
        case FSMST_Q0:
            //stepcnt=1;

            if (rsymb != '_') {
                nextaddrtm = curraddrtm - 1;
            }
            else {
                fsmstate = FSMST_Q1;
                nextaddrtm = curraddrtm + 1;
            }
        }

    break;
}

```

```

        case FSMST_Q1:
            if(rsymb == '_')
            { // _,R
                fsmstate = FSMST_Q6;
            }

        if (rsymb == 'x')
        {
            nextaddrtm = curraddrtm + 1;
        }

        if (rsymb == 'a')
        {
            nextaddrtm = curraddrtm + 1;
            fsmstate = FSMST_Q3;

            *wrsymb = 'x';
        }

        if (rsymb == 'b')
        {
            nextaddrtm = curraddrtm + 1;
            fsmstate = FSMST_Q2;

            *wrsymb = 'x';
        }

        break;

        case FSMST_Q2:
        if(rsymb == '_')
            { // _,R
                nextaddrtm = curraddrtm + 1;
                fsmstate = FSMST_Q4;
            }

        if (rsymb == 'b' || rsymb == 'x')
        {
            nextaddrtm = curraddrtm + 1;
        }

        if (rsymb == 'a')
        {
            nextaddrtm = curraddrtm - 1;
            fsmstate = FSMST_Q0;
            *wrsymb = 'x';
        }

        break;

    case FSMST_Q3:

```

```

    if(rsymb == '_')
        { // _,_,R
            nextaddrtm = curraddrtm + 1;
            fsmstate = FSMST_Q5;
        }

    if (rsymb == 'a' || rsymb == 'x')
    {
        nextaddrtm = curraddrtm + 1;
    }

    if (rsymb == 'b')
    {
        nextaddrtm = curraddrtm - 1;
        fsmstate = FSMST_Q0;
        *wrsymb = 'x';
    }

    break;

case FSMST_Q4:
    if(rsymb == '_')
        { // _,_,R
            *wrsymb = 'b';
            fsmstate = FSMST_Q6;
        } else {
            *wrsymb = 'e';
            fsmstate = FSMST_Q6;
        }

    break;

case FSMST_Q5:
    if(rsymb == '_')
        { // _,_,R
            *wrsymb = 'a';
            fsmstate = FSMST_Q6;
        }
    else {
        *wrsymb = 'e';
        fsmstate = FSMST_Q6;
    }

    break;

    case FSMST_Q6:
        stepcnt=-1;

```



```
                break;
            }
        return stepcnt;
    }
```

Выводы

Была реализована одновременная работа двух машин Тьюринга для определения самой часто встречающейся буквы в слове. Память машин была разделена по 8 байт, адреса памяти располагались для машин последовательно. Реализовано “вмешательство” в работу машин по нажатию на кнопки K1-K16 добавлением пустого символа в выбранный адрес.